

Thực hành mã bất đồng bộ

1 Thư viện asyncio

Thư viện asyncio trong Python giúp ta viết các chương trình xử lý nhiều việc cùng lúc bằng cách sử dụng hai từ khóa chính là `async` và `await`. Cốt lõi của phương pháp này xoay quanh các "công việc có thể tạm dừng"(coroutine) và một "người quản lý"(vòng lặp sự kiện - event loop) để sắp xếp lịch trình thực thi cho chúng. Điều hay nhất ở đây là mô hình I/O bất đồng bộ chỉ sử dụng một luồng và một bộ xử lý duy nhất để thực hiện "đa tác vụ hợp tác", nghĩa là các công việc sẽ tự giác nhường quyền kiểm soát cho nhau khi chúng rảnh rỗi hoặc phải chờ đợi thay vì bất toàn bộ hệ thống phải đứng im. Nhờ vậy, chương trình của ta có thể quản lý nhiều công việc "đồng thời" trong cùng một giai đoạn, nhưng không thực sự chạy "song song" tại cùng một tích tắc, tạo nên một hướng tiếp cận độc lập, nhẹ nhàng và linh hoạt hơn hẳn so với kỹ thuật đa luồng hay đa xử lý truyền thống.

1.1 Coroutine và hàm Coroutine

Cốt lõi của I/O bất đồng bộ là khái niệm của coroutine, về bản chất là một đoạn mã đặc biệt (đối tượng) có khả năng tự tạm dừng hoạt động trong một khoảng thời gian (thường là để chờ dữ liệu) và sẽ tự động tiếp tục chạy sau đó. Trong lúc đang tạm nghỉ, coroutine này sẽ chủ động nhường lại quyền điều khiển cho vòng lặp sự kiện, giúp hệ thống tranh thủ khoảng thời gian trống đó để xử lý ngay một coroutine (công việc) khác.

Đối tượng của coroutine là kết quả trả về từ hàm coroutine, còn được biết đến là hàm bất đồng bộ, ta định nghĩa nó với một hàm khởi tạo `async def`.

Trước khi viết hàm `async def` một lần nữa, hãy chú ý tới ví dụ sau về chương trình chạy theo luồng đồng bộ:

```
# Ví dụ 1: Dùng module time bình thường
import time
def count():
    print("One")
    time.sleep(1)
    print("Two")
    time.sleep(1)

def counting():
    for _ in range(3):
        count()

if __name__ == "__main__":
    start = time.perf_counter()
    counting()
    elapsed = time.perf_counter() - start
    print(f"Chương trình thực hiện với thời gian là {elapsed:0.2f} giây.")
```

Hình 1: Chương trình đồng bộ chạy bằng module `time`

Trong đó:

- Thư viện `time` được sử dụng để dùng hàm `sleep()` trong Python. Đây là một **lệnh chặn**, khi Python thông dịch dòng này, nó sẽ ngừng luồng của hệ thống HDH đang hoạt động trong 1 giây, Python lúc này hoàn toàn "đứng yên".
- Hàm `range()` được dùng với tham số gạch dưới (underscore) để không cần phải theo dõi với một biến khác. Nó sẽ thực hiện dừng 2 giây trước khi thực hiện một vòng lặp khác (tổng cộng 3 lần).
- Hàm `perf_counter()` được dùng để đo thời gian của hiệu suất code, chính xác tới millisecond. Biến `elapsed` cuối cùng chỉ để lấy thời gian hoàn thành trừ thời gian bắt đầu để xem đoạn code thực hiện trong bao lâu.

Nếu chạy chương trình, ta sẽ có kết quả sau:

```
One
Two
One
Two
One
Two
Chương trình thực hiện với thời gian là 6.00 giây.
```

Hình 2: Kết quả của chương trình với module `time`

Ta mất đúng (xấp xỉ) 6 giây để hoàn thành chương trình như mong đợi.

Nếu ta dùng module `asyncio` để thay thế như sau:

```

#Ví dụ 2: Dùng module asyncio
import asyncio

async def count():
    print("One")
    await asyncio.sleep(1)
    print("Two")
    await asyncio.sleep(1)

async def counting():
    await asyncio.gather(count(), count(), count())

if __name__ == "__main__":
    import time
    start = time.perf_counter()
    asyncio.run(counting())
    elapsed = time.perf_counter() - start
    print(f"Chương trình đã thực hiện với thời gian là {elapsed:0.2f} giây.")

```

Hình 3: Chương trình bất đồng bộ chạy bằng module `asyncio`

Trong đó:

- Hàm `async def count()` với từ khóa `async` sẽ biến đổi mọi hàm bình thường thành hàm Coroutine. Gọi hàm **sẽ không thực hiện code nữa**, mà xây nên một đối tượng Coroutine có thể được dùng để chuẩn bị cho quá trình điều phối công việc sau đó.
- Từ khóa `await` sử dụng để thay thế cho hàm `time.sleep()` thông thường, nó có nghĩa là "Dừng ở đây 1 giây, còn vòng lặp sự kiện của chương trình sẽ làm công việc khác khi đang đợi". CPU ở đây sẽ không bị chặn như `time.sleep()`.
- Hàm coroutine `asyncio.gather()` sẽ chạy 3 thể hiện đối tượng `count()` đồng thời.
- Sử dụng thêm hàm `asyncio.run()` để khởi động vòng lặp sự kiện (event loop) và thực thi hàm `counting()`.

Kết quả theo lý thuyết khi chạy sẽ là:

1. Cả 3 hàm đều bắt đầu chạy và lần lượt in ra ba chữ "One" gần như cùng một lúc.
2. Sau khi in xong chữ "One", cả 3 hàm đều chạm tới dòng lệnh `await asyncio.sleep(1)` và đồng loạt báo tạm dừng nhường quyền 1 giây. Vòng lặp sự kiện ghi nhận điều này và chuyển qua trạng thái chờ.
3. Đúng 1 giây sau, vòng lặp sự kiện đánh thức cả 3 hàm dậy. Cả 3 lại tiếp tục chạy và in ra ba chữ "Two".

Kết quả của chương trình chạy sẽ là:

```

One
One
One
Two
Two
Two
Chương trình đã thực hiện với thời gian là 2.00 giây.

```

Hình 4: Kết quả của chương trình với module `asyncio`

Vì 3 khoảng thời gian nghỉ 1 giây được diễn ra 'chồng' lên nhau trong cùng một lúc, tổng thời gian để hoàn thành chương trình chỉ mất vỏn vẹn 2 giây thay vì 6 giây như cách chạy xếp hàng đợi nhau ở phương pháp đồng bộ.

2 Từ khóa `async` và `await`

Các từ khóa `async` và `await` cùng với hàm coroutine có thể như sau:

1. Cấu trúc cú pháp `async def` **giới thiệu một hàm coroutine hoặc một generator bất đồng bộ**. Bất cứ khi nào ta viết `async def` trước một hàm, ta đang tạo ra một quy trình làm việc đa nhiệm (coroutine). Nó khai báo rằng công việc này có thể được tạm dừng giữa chừng để làm việc khác.
2. Cấu trúc cú pháp `async with` và `async for` **lần lượt tạo các câu lệnh with** và vòng lặp `for` bất đồng bộ. Đây là hành động nhường quyền.
3. Từ khóa `await` **tạm dừng** việc thực thi của coroutine bao quanh và **trả lại quyền điều khiển** cho **vòng lặp sự kiện**. Khi một hàm `g()` đang chạy và gặp lệnh `await f()`, hệ thống sẽ hiểu là: "Hãy dừng công việc G lại, chờ công việc F hoàn thành, và trong lúc chờ F, hãy rảnh tay đi làm việc khác".

Để làm rõ hơn một chút nữa, hãy cho một đoạn code Python như sau:

```

async def g():
    result = await f() #Pause and come back to g() when f() returns
    return result

```

Khi Python gặp lệnh `await f()` trong khối của hàm coroutine `g()`, từ khóa `await` sẽ nói với vòng lặp sự kiện rằng "dừng sự thực thi của hàm `g()` cho đến khi hàm `f()` được trả về, trong thời gian đó hãy làm cái gì đó khác". Ngoài ra còn có **một bộ quy tắc nghiêm ngặt về thời điểm và cách có thể sử dụng `async` và `await`**. Những quy tắc này hữu ích cho dù vẫn đang học cú pháp hay đã quen thuộc với việc sử dụng `async` và `await`:

- Sử dụng cấu trúc `async def`, ta có thể định nghĩa một hàm coroutine. Nó có thể sử dụng `await`, `return` hoặc `yield`. Tuy nhiên, để gọi

một quy trình coroutine khác thực thi, ta bắt buộc phải dùng `await` để chờ kết quả của nó, hoặc ném nó cho vòng lặp sự kiện tự sắp xếp. Tất cả đều là tùy chọn:

- `await`, `return` hoặc cả hai có thể được sử dụng trong các hàm coroutine thông thường. Để gọi một hàm coroutine, phải sử dụng `await` để nhận kết quả hoặc chạy trực tiếp nó trong một vòng lặp sự kiện.
- `yield` được sử dụng trong một hàm `async def` tạo ra một generator bất đồng bộ. Để lặp lại generator này nhằm thực hiện lấy dữ liệu, có thể sử dụng một vòng lặp `async for` hoặc một list comprehension.
- `async def` không được sử dụng `yield from`, điều này sẽ gây ra lỗi `SyntaxError`.

- Sử dụng `await` bên ngoài một hàm `async def` cũng gây ra lỗi cú pháp `SyntaxError` nên chỉ có thể sử dụng `await` trong phần thân của coroutine. Nó giống như ta không thể bảo một nhân viên bình thường (hàm `def` truyền thống) thực hiện kỹ năng "tạm dừng để làm việc khác"(`await`). Việc sử dụng `await` bên ngoài một hàm `async def` sẽ lập tức bị hệ thống báo lỗi cú pháp (`SyntaxError`).

Dưới đây là một số ví dụ ngắn gọn tóm tắt các quy tắc này:

```
async def f(x):
    y = await z(x) # Okay - 'await' and 'return'
    # allowed in coroutines
    return y

async def g(x):
    yield x # Okay - this is an async generator

async def m(x):
    yield from gen(x) # No - SyntaxError

def n(x):
    y = await z(x) # No - SyntaxError (no async def here)
    return y
```

Cuối cùng, khi ta sử dụng `await f()`, điều kiện bắt buộc là `f()` phải là một đối tượng có thể được chờ đợi (awaitable object), đó có thể là một coroutine khác hoặc một đối tượng định nghĩa phương thức đặc biệt `__await__()` trả về một iterator (là một đối tượng giúp ta duyệt (lặp) qua từng phần tử của một tập hợp như list, array, set, map... mà không cần biết cách tập hợp đó được lưu trữ bên trong như thế nào). Trong hầu hết các trường hợp, chỉ cần quan tâm đến coroutine.

Dưới đây là một trường hợp ví dụ nữa mà `async I/O` giảm thời gian chờ đợi. Bây giờ cho rằng có hàm coroutine gọi là `makerandom()` sản sinh ra các số nguyên từ 1 đến 10 và trả về một trong các số đó khi chúng vượt quá ngưỡng, trong ví

dụ này, ta sẽ chạy hàm này bất đồng bộ đến 3 lần, mỗi lần thì được dùng một màu khác nhau.

```
import asyncio
import random
COLORS = (
    "\033[0m", # End of color
    "\033[36m", # Cyan
    "\033[31m", # Red
    "\033[35m", # Magenta
)

async def main():
    return await asyncio.gather(
        makerandom(1, 9),
        makerandom(2, 8),
        makerandom(3, 8),
    )

async def makerandom(delay, threshold=6):
    color = COLORS[delay]
    print(f"{color}Initiated makerandom({delay}).")
    while (number := random.randint(0, 10)) <= threshold:
        print(f"{color}makerandom({delay}) == {number} too low; retrying.")
        await asyncio.sleep(delay)
    print(f"{color}--> Finished: makerandom({delay}) == {number} + {COLORS[0]}")
    return number

if __name__ == "__main__":
    random.seed(444)
    r1, r2, r3 = asyncio.run(main())
    print()
    print(f"r1: {r1}, r2: {r2}, r3: {r3}")
```

Hình 5: Chương trình bất đồng bộ tạo số ngẫu nhiên chạy bằng async I/O

Trong đó, một số giải thích chi tiết của đoạn mã trên là như sau:

- Tại hàm `async def main()`, thực hiện 3 thể hiện coroutine cùng một lúc, hàm `makerandom()` đầu tiên thực hiện lại sau mỗi giây, hàm `makerandom()` thứ hai thực hiện lại sau 2 giây và cứ tiếp tục đến thế, cả ba hàm đều tìm số lớn hơn 8 (trừ hàm đầu tiên số lớn hơn 9).
- Gán các màu dưới dạng hex từ biến `COLORS` vào tham số `delay` để ta có thể nhìn thấy tác vụ nào đang thực hiện.
- Toán tử Walrus (`:=`) sẽ lấy hàm tạo số nguyên ngẫu nhiên từ 0 đến 10 rồi gán vào biến `number` để so sánh lập tức với ngưỡng (biến `threshold`) xem có lớn hơn hay không. Miễn là nó chưa lớn hơn thì vòng lặp sẽ tiếp tục chạy.
- Từ khóa `await` cho lần gọi hàm `asyncio.sleep(delay)` là để dừng bất đồng bộ cho tác vụ nào chưa thành công, ví dụ như nếu hàm `makerandom()` đầu tiên cho ra là 4, in ra là `too low` thì sẽ trao quyền điều khiển cho vòng lặp sự kiện trong 1 giây (cơ bản bị ngừng trong 1 giây). Vòng lặp sự kiện sẽ đi qua tác vụ thứ hai (hàm `makerandom()` thứ hai) để xem nếu tác vụ thứ hai và thứ ba sẵn sàng để tạo ngẫu nhiên tiếp một số hay chưa.
- Sử dụng `random.seed(444)` để thực hiện một quá trình toán học, ép sự ngẫu nhiên đó được thực thi lại mỗi lần. Rồi cuối cùng là lấy 3 biến `r1` đến `r3` để gán cho 3 giá trị nguyên trả về rồi in ra kết quả.

Kết quả từ đoạn mã này có thể là như sau:

```
$ python3 asyncioEx2.py
Initiated makerandom(1).
makerandom(1) == 4 too low; retrying.
Initiated makerandom(2).
makerandom(2) == 4 too low; retrying.
Initiated makerandom(3).
makerandom(3) == 0 too low; retrying.
makerandom(1) == 4 too low; retrying.
makerandom(2) == 7 too low; retrying.
makerandom(1) == 4 too low; retrying.
makerandom(3) == 4 too low; retrying.
makerandom(1) == 8 too low; retrying.
--> Finished: makerandom(2) == 10
makerandom(1) == 7 too low; retrying.
makerandom(1) == 8 too low; retrying.
makerandom(3) == 4 too low; retrying.
makerandom(1) == 7 too low; retrying.
makerandom(1) == 1 too low; retrying.
makerandom(1) == 6 too low; retrying.
--> Finished: makerandom(3) == 9
makerandom(1) == 3 too low; retrying.
makerandom(1) == 9 too low; retrying.
makerandom(1) == 7 too low; retrying.
--> Finished: makerandom(1) == 10

r1: 10, r2: 10, r3: 9
```

Hình 6: Kết quả chương trình bất đồng bộ tạo số ngẫu nhiên chạy bằng async I/O

Chương trình này định nghĩa coroutine `makerandom()` và chạy nó đồng thời với ba đầu vào khác nhau, kết quả trên có thể cho biết quá trình thực hiện là như sau:

1. Giây thứ 0: Hàm 1 bắt đầu chạy, bốc số hũt, nó lập tức ngừng 1 giây. Do hàm 1 bị ngừng, event loop lập tức nhường quyền cho hàm 2. Hàm 2 bốc số hũt, ngừng 2 giây. Event loop lại lập tức nhường quyền cho hàm 3. Hàm 3 bốc số hũt, ngừng 3 giây. Tất cả thực hiện rất nhanh, không đáng kể về mặt thời gian.
2. Giây thứ 1: Hàm 1 hết thời gian 1 giây, nó thức dậy. Event loop cho nó chạy tiếp. Theo kết quả thì nó lại bốc hũt, nó tiếp tục ngừng thêm 1 giây nữa (thức dậy ở giây thứ 2). Trong lúc này, hàm 2 và hàm 3 vẫn đang ngừng (vì chúng ngừng với độ trễ 2s và 3s).
3. Giây thứ 2: Lúc này có 2 sự kiện cùng xảy ra là hàm 1 vừa ngừng xong lần thứ hai ($1s + 1s = 2s$) và hàm 2 cũng vừa xong lần ngừng đầu tiên

(2s). Cả hai cùng thức dậy. Event loop sẽ chia thời gian (tồn 0 giây) để cả hai bốc số. Ai hụt nữa thì tiếp tục ngừng.

4. Giấy thứ 3: Đến lúc này, hàm 3 mới hết thời hạn ngừng 3 giây đầu tiên và thức dậy để bốc số lần thứ hai.

Hầu hết các chương trình sẽ bao gồm các coroutine nhỏ, có tính module và một hàm bao bọc dùng để liên kết các coroutine nhỏ hơn với nhau. Trong hàm `main()`, ta **tập hợp** ba tác vụ. Ba lần gọi hàm `makerandom()` chính là **tập hợp các tác vụ** của ta.

Mặc dù việc tạo số ngẫu nhiên trong ví dụ này là một tác vụ CPU-bound, nhưng tác động của nó là không đáng kể. Phương thức `asyncio.sleep()` mô phỏng một tác vụ I/O bound và cho thấy rằng **chỉ những tác vụ I/O bound hoặc không chặn mới được hưởng lợi từ mô hình I/O bất đồng bộ**.

3 Vòng lặp sự kiện Async I/O

Trong lập trình bất đồng bộ, vòng lặp sự kiện biết như là **một vòng lặp vô tận** mà quản lý các coroutine, xem phản hồi cho những cái gì đang rảnh, và tìm đến những thứ mà có thể được thực thi trong thời gian rảnh đó. Nó có thể "đánh thức" một coroutine rảnh khi mà coroutine đang đợi đó có thể sử dụng.

Cách mà được khuyên sử dụng vòng lặp sự kiện trong Python hiện đại là sử dụng hàm `asyncio.run()`, hàm này chịu trách nhiệm cho việc lấy vòng lặp sự kiện, thực thi tác vụ cho đến khi chúng xong và đóng vòng lặp. Ta không thể gọi hàm này khi mà vòng lặp sự kiện bất đồng bộ đang chạy trên hệ thống.

Cũng có thể lấy thể hiện của vòng lặp sự kiện đang chạy với phương thức `get_running_loop()` như sau:

```
loop = asyncio.get_running_loop()
```

Nếu cần tương tác với vòng lặp sự kiện trong một chương trình Python, câu lệnh là một cách tốt để thực hiện điều đó. Đối tượng vòng lặp hỗ trợ kiểm tra nội tại bằng `.is_running()` và `.is_closed()`. Điều này có thể hữu ích **khi muốn lên lịch một hàm gọi lại bằng cách truyền vòng lặp làm đối số** chẳng hạn. Lưu ý rằng `get_running_loop()` sẽ ném ra ngoại lệ `RuntimeError` nếu không có vòng lặp sự kiện nào đang chạy.

Điều quan trọng hơn là hiểu được những gì diễn ra bên dưới bề mặt của vòng lặp sự kiện. Dưới đây là một vài điểm đáng nhấn mạnh:

- Một hàm coroutine sẽ nằm im bất động và vô tác dụng nếu không được gắn vào vòng lặp sự kiện. Đó là lý do nếu ta chỉ gọi hàm `main()` trống không, hệ thống chỉ tạo ra một gói coroutine chứ không hề chạy dòng lệnh bên trong đó. Ta bắt buộc phải bọc nó bằng `asyncio.run()` hoặc dùng từ khóa `await`.

- Theo mặc định, một vòng lặp sự kiện bất đồng bộ chạy trong một luồng duy nhất và trên một lõi CPU duy nhất. Trong hầu hết các ứng dụng `asyncio`, sẽ chỉ có một vòng lặp sự kiện, thường là trong luồng chính. Việc chạy nhiều vòng lặp sự kiện trong các luồng khác nhau về mặt kỹ thuật là có thể, nhưng không thường cần thiết hoặc được khuyến nghị.
- Vòng lặp sự kiện có thể cắm thêm (**pluggable**), tức là ta không bị ép phải dùng một kiểu quản lý cố định. Tùy vào môi trường mà Python sẽ cử ra người phù hợp nhất (ví dụ hệ thống Unix thường mặc định dùng `SelectorEventLoop`, còn Windows dùng `ProactorEventLoop` để hỗ trợ tốt hơn). Thậm chí, ta có thể cài thêm các thư viện của bên thứ ba (như `uvloop`) để làm vòng lặp sự kiện thay thế vì chúng xử lý tốc độ nhanh hơn.

Về điểm đầu tiên, nếu có một coroutine đang chờ các coroutine khác, thì việc gọi nó một cách riêng lẻ sẽ không có nhiều tác dụng:

```
>>> import asyncio

>>> async def main():
...     print("Hello...")
...     await asyncio.sleep(1)
...     print("World!")
...

>>> routine = main()
>>> routine
<coroutine object main at 0x1027a6150>
```

Trong ví dụ này, việc gọi trực tiếp hàm `main()` sẽ trả về một đối tượng coroutine mà không thể sử dụng độc lập. Ta cần sử dụng `asyncio.run()` để lên lịch cho coroutine `main()` thực thi trong vòng lặp sự kiện:

```
>>> asyncio.run(routine)
Hello...
World!
```

Thông thường, ta sẽ bao bọc coroutine `main()` của mình trong một lệnh gọi `asyncio.run()`. Ta có thể thực thi các coroutine cấp thấp hơn bằng `await`.

Cuối cùng, việc vòng lặp sự kiện có thể cắm thêm các module có nghĩa là có thể sử dụng bất kỳ triển khai vòng lặp sự kiện nào đang hoạt động, và điều đó không liên quan đến cấu trúc coroutine của ta. Gói `asyncio` đi kèm với hai triển khai vòng lặp sự kiện khác nhau.

4 asyncio REPL

Từ Python 3.8, module `asyncio` bao gồm một shel tương tác chuyên biệt hóa gọi là asyncio REPL. Môi trường này giúp ta sử dụng lệnh `await` với cấp cao nhất, **không cần phải đóng gói** mã của ta trong một lần gọi `asyncio.run()`. Công cụ này sẽ thiết lập, thử nghiệm và gỡ lỗi `asyncio` trên Python.

Để thực thi REPL, ta có thể chạy lệnh dưới đây:

```
python -m asyncio
asyncio REPL 3.14.6 (main, Jun 10 2026, 18:54:31)
[GCC 15.2.0] on linux
Use "await" directly instead of "asyncio.run()".
Type "help", "copyright", "credits" or "license"
for more information.
>>> import asyncio
>>>
```

Khi mà đã có prompt ba dấu lớn hơn, ta có thể bắt đầu chạy mã bất đồng bộ ở đây, như ví dụ dưới đây:

```
>>> import asyncio

>>> async def main():
...     print("Hello...")
...     await asyncio.sleep(1)
...     print("World!")
...

>>> await main()
Hello...
World!
```

Ví dụ này hoạt động tương tự như ví dụ trong phần trước. Tuy nhiên, thay vì chạy hàm `main()` bằng `asyncio.run()`, sử dụng `await` trực tiếp.

5 Các quy luật lập trình I/O bất đồng bộ thường thấy

5.1 Xích coroutine - Coroutine chaining

Một đặc điểm quan trọng của coroutine là có thể xích chúng lại với nhau. Hãy nhớ rằng một coroutine có thể được chờ đợi, vì vậy một coroutine khác có thể chờ đợi nó bằng từ khóa `await`. Điều này giúp dễ dàng chia chương trình của ta thành các coroutine nhỏ hơn, dễ quản lý và có thể tái sử dụng.

Ví dụ dưới đây mô phỏng quy trình hai bước để lấy thông tin về người dùng. Bước đầu tiên lấy thông tin người dùng, và bước thứ hai lấy các bài đăng mà người dùng đó đã xuất bản:

```
import asyncio
import random
import time

async def main():
    user_ids = [1, 2, 3]
    start = time.perf_counter()
    await asyncio.gather(
        *(get_user_with_posts(user_id) for user_id in user_ids)
    )
    end = time.perf_counter()
    print(f"Total time: {end - start:.2f} seconds")

async def get_user_with_posts(user_id):
    user = await fetch_user(user_id)
    await fetch_posts(user)

async def fetch_user(user_id):
    delay = random.uniform(0.5, 2.0)
    print(f"User coro: fetched user with {user_id}...")
    await asyncio.sleep(delay)
    user = {"id": user_id, "name": f"User{user_id}"}
    print(f"User coro: fetched user with {user_id=} (done in {delay:.1f}s).")
    return user

async def fetch_posts(user):
    delay = random.uniform(0.5, 2.0)
    print(f"Post coro: retrieving posts for {user['name']}...")
    await asyncio.sleep(delay)
    posts = [f"Post {i} by {user['name']}" for i in range(1, 3)]
    print(
        f"Post coro: got {len(posts)} posts by {user['name']}"
        f" (done in {delay:.1f}s):"
    )
    for post in posts:
        print(f" - {post}")

if __name__ == "__main__":
    random.seed(444)
    asyncio.run(main())
```

Hình 7: Đoạn mã mô phỏng quy trình hai bước từ ví dụ

Trong đoạn mã trên, ta có thể thấy có hai coroutine lớn là `fetch_user()` và `fetch_post()`. Cả hai giả lập một lời gọi qua mạng với độ trễ ngẫu nhiên sử dụng `asyncio.sleep()` với biến `delay` mang giá trị ngẫu nhiên, phân tích chi tiết đoạn code này như sau:

1. `fetch_user(user_id)`: Tải hồ sơ (profile) của người dùng.

- Hàm này nhận vào ID của người dùng `user_id`.
- `random.uniform(0.5, 2.0)` sẽ giả lập độ trễ của một mạng không đồng đều (từ 0.5 đến 2 giây). Vì mỗi người dùng có thể ở các vị trí khác nhau.
- `await asyncio.sleep(delay)` sẽ báo với hệ thống là: "Tôi đang bận tải hồ sơ, xin hãy chờ ở đây một lát" (hệ thống sẽ đi làm việc khác).
- `user = {'id': user_id, 'name': f'User{user_id}'}` sẽ thực thi sau khi ngủ xong, nó tự tạo ra một tự điển chứa thông tin người dùng (payload giả) rồi `return user` trả kết quả về.

2. Hàm `fetch_posts(user)`: Tải bài đăng (post) từ người dùng xác định.
 - Hàm này bắt buộc phải nhận vào kết quả `user` từ hàm `fetch_user` mới chạy được.
 - Nó sẽ trích xuất tên người dùng (`user['name']`). Nếu hàm trước bị lỗi không có dữ liệu, hàm này sẽ báo lỗi `KeyError` ngay lập tức vì nó phụ thuộc chặt chẽ vào tiền xử lý.
 - Nó lại tiếp tục tạo `delay` và `await asyncio.sleep(delay)` để mô phỏng việc mất thời gian truy vấn vào cơ sở dữ liệu để lấy các bài đăng.
 - Cuối cùng, nó trả về một danh sách các bài đăng giả lập.
3. Hàm `get_user_with_posts(user_id)`: Ông dẫn xâu chuỗi.
 - Đây là dây chuyền ghép Bước 1 và Bước 2 lại với nhau:


```
user = await fetch_user(user_id)
await fetch_posts(user)
```
 - Mọi thứ bên trong hàm này phải chạy tuần tự cho một ID cụ thể. Nó chặn chính nó (không chuyển thẳng đến tải bài viết được), nhưng nó không hề chặn toàn bộ máy chủ.
4. Tại hàm `main()` tham số trong `asyncio.gather()` sử dụng toán tử mở gói (Unpacking operator - `*`) để mở danh sách một cách động, đổi thành `asyncio.gather(get_user_with_posts(1), get_user_with_posts(2), get_user_with_posts(3))`.
5. `random.seed(444)` tiếp tục được sử dụng để tính toán bộ cục số chấm phẩy động ngẫu nhiên cho mỗi lần chạy.

Từ các giải thích trên, cấu trúc luồng đồng thời cụ thể sẽ bao gồm:

1. Bên trong một quy trình xử lý riêng lẻ (luồng dọc): Quá trình thực thi diễn ra tuần tự. Bài đăng của Người dùng 1 phải chờ hồ sơ của Người dùng 1 được cập nhật.
2. Trên nhiều đường dẫn xử lý (luồng ngang): Quá trình thực thi diễn ra đồng thời. Hồ sơ Người dùng 1, Người dùng 2 và Người dùng 3 đều được lấy cùng một lúc.

Về kết quả, nó sẽ là như sau:

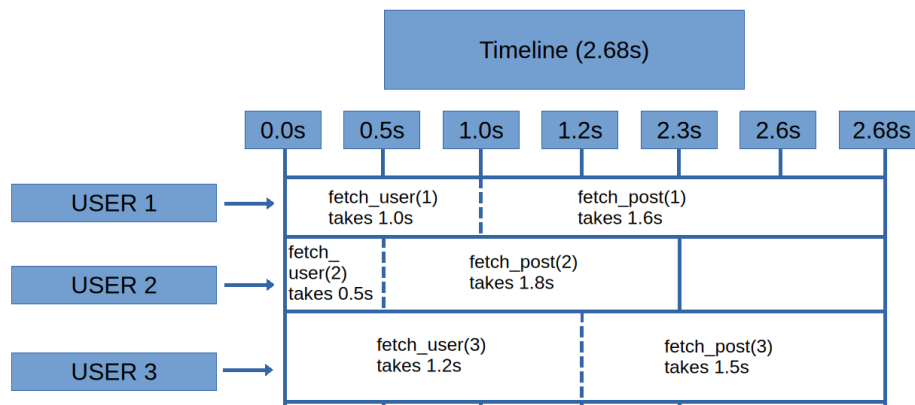
```

$ python3 asyncioEx3.py
User coro: fetched user with user_id=1...
User coro: fetched user with user_id=2...
User coro: fetched user with user_id=3...
User coro: fetched user with user_id=2 (done in 0.5s).
Post coro: retrieving posts for User2...
User coro: fetched user with user_id=1 (done in 1.0s).
Post coro: retrieving posts for User1...
User coro: fetched user with user_id=3 (done in 1.2s).
Post coro: retrieving posts for User3...
Post coro: got 2 posts by User2 (done in 1.8s):
- Post 1 by User2
- Post 2 by User2
Post coro: got 2 posts by User1 (done in 1.6s):
- Post 1 by User1
- Post 2 by User1
Post coro: got 2 posts by User3 (done in 1.5s):
- Post 1 by User3
- Post 2 by User3
Total time: 2.68 seconds

```

Hình 8: Kết quả đoạn mã mô phỏng quy trình hai bước từ ví dụ

Dưới đây là lưu đồ để biểu diễn dòng thời gian diễn ra quá trình:



Hình 9: Lưu đồ mô phỏng quy trình hai bước từ ví dụ

Tại lưu đồ, bước thực thi là như sau:

- Giây 0: Cả 3 luồng công việc (User 1, User 2, User 3) đều đồng loạt bắt đầu `fetch_user`. Cả 3 đều gọi lệnh tạm ngưng để giả lập tải mạng. User 1 bị lag mất 1.0s, User 2 lag 0.5s, User 3 lag 1.2s. Hệ thống đi vào trạng thái chờ chung.
- Giây 0.5: User 2 tải hồ sơ xong sớm nhất (chỉ tốn 0.5s). Luồng dọc của User 2 lập tức chuyển sang Bước 2 là `fetch_post(2)`. Lệnh tải bài viết này mô phỏng tốn tận 1.8s. Vậy là User 2 lại tiếp tục rơi vào trạng thái chờ. (Trong lúc này, User 1 và User 3 vẫn đang rảnh chờ tải hồ sơ).
- Giây 1.0: User 1 hoàn tất việc tải hồ sơ. Luồng của User 1 lập tức chuyển sang `fetch_post(1)`. Lệnh này mô phỏng mất 1.6s. User 1 lại đi chờ. (Lúc này, User 3 vẫn đang đợi, còn User 2 thì đang tải bài viết được 0.5s).

4. Giấy 1.2: User 3 cũng tải hồ sơ xong. Nó bước vào `fetch_post(3)` tốn 1.5s và cũng đi chờ nốt.
5. Giấy 2.3: User 2 đã chờ xong 1.8s của `fetch_post(2)`. Nó chính thức hoàn thành toàn bộ công việc và in ra màn hình 2 bài viết của mình.
6. Giấy 2.6: User 1 chờ xong 1.6s tải bài viết. Nó hoàn thành toàn bộ công việc.
7. Giấy 2.68: User 3 hoàn thành nốt công việc tải bài viết và in ra kết quả cuối cùng. Chương trình kết thúc.

5.2 Tích hợp Coroutine và hàng đợi

Gói `asyncio` cung cấp một vài lớp giống như hàng đợi, được thiết kế tương tự như các lớp trong module `queue`. Trong các ví dụ cho đến nay chưa cần đến cấu trúc hàng đợi. Trong ví dụ trước, mỗi tác vụ được thực hiện bởi một coroutine, mà có thể xâu chuỗi với các coroutine khác để truyền dữ liệu từ coroutine này sang coroutine khác.

Một cách tiếp cận khác là sử dụng các nhà sản xuất (producer) thêm các mục vào hàng đợi. Mỗi producer có thể thêm nhiều mục vào hàng đợi vào những thời điểm khác nhau, ngẫu nhiên và không báo trước. Sau đó, một nhóm người tiêu dùng (consumers) sẽ lấy các mục từ hàng đợi khi chúng xuất hiện, một cách tham lam và không cần chờ bất kỳ tín hiệu nào khác.

Trong thiết kế này, không có sự xâu chuỗi giữa các nhà sản xuất và người tiêu dùng. Người tiêu dùng không biết số lượng nhà sản xuất, và ngược lại.

Mỗi nhà sản xuất hoặc người tiêu dùng cần một khoảng thời gian thay đổi để thêm và xóa các mục khỏi hàng đợi. Hàng đợi đóng vai trò như một bộ lọc thông lượng có thể giao tiếp với các nhà sản xuất và người tiêu dùng mà không cần họ nói chuyện trực tiếp với nhau. Cũng sử dụng mô hình đăng tải bài đăng, có được ví dụ với đoạn code sau:

```

import asyncio
import random
import time

async def main():
    queue = asyncio.Queue()
    user_ids = [1, 2, 3]

    start = time.perf_counter()
    await asyncio.gather(
        producer(queue, user_ids), *(consumer(queue) for _ in user_ids),
    )
    end = time.perf_counter()
    print(f"Total time: {end - start:.2f} seconds.")

async def producer(queue, user_ids):
    async def fetch_user(user_id):
        delay = random.uniform(0.5, 2.0)
        print(f"Producer: fetching user by {user_id}...")
        await asyncio.sleep(delay)
        user = {"id": user_id, "name": f"User{user_id}"}
        print(f"Producer: fetched user with {user_id=} (done in {delay:.1f}s)")
        await queue.put(user)

    await asyncio.gather(*(fetch_user(uid) for uid in user_ids))
    for _ in range(len(user_ids)):
        await queue.put(None) # Sentinels for consumers to terminate

async def consumer(queue):
    while True:
        user = await queue.get()
        if user is None:
            break
        delay = random.uniform(0.5, 2.0)
        print(f"Consumer: retrieving posts for {user['name']}...")
        await asyncio.sleep(delay)
        posts = [f"Post {i} by {user['name']}" for i in range(1, 3)]
        print(
            f"Consumer: got {len(posts)} posts by {user['name']}"
            f" (done in {delay:.1f}s):"
        )
        for post in posts:
            print(f" - {post}")

if __name__ == "__main__":
    random.seed(444)
    asyncio.run(main())

```

Hình 10: Đoạn mã mô phỏng quy trình đăng bài đăng bằng hàng đợi bất đồng bộ

Chi tiết đoạn mã có thể được phân tích như sau:

1. `queue = asyncio.Queue()` là một container đệm bất đồng bộ với quy tắc FIFO (First In First Out). Được thiết kế cụ thể để cho các coroutine có thể ghi (`put` - đặt vào) và đọc (`get` - lấy ra) mà không va chạm với nhau.
2. Hàm `asyncio.gather()` sẽ khởi động một coroutine `Producer` và 3 coroutine `Consumer` cùng một lúc. Tổng lại sẽ có 4 thao tác độc lập chạy cùng lúc.
3. `async def producer(...)` là hàm chính. Hàm này sẽ liên tục gọi `fetch_user` để tải hồ sơ. Khi tải xong một người, hàm dùng lệnh `await queue.put(user)` để đặt hồ sơ lên hàng đợi. Hành động đặt lên kệ tốn khoảng thời gian gần

như bằng 0, nên hàm có thể thực hiện việc khác ngay (ở đây thì giống như là tải về profile trên mạng vậy).

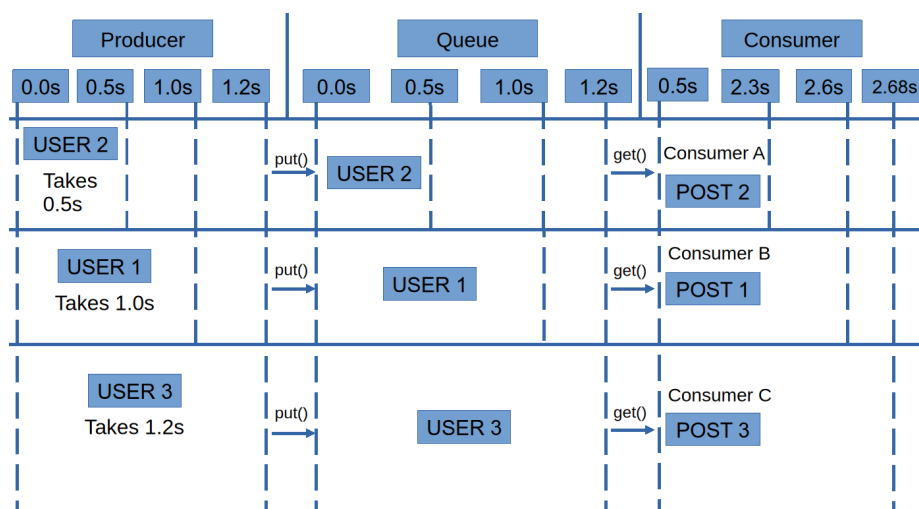
4. `None` được gọi là một Sentinel value hay Poison Pill, Ở cuối hàm producer, sau khi đã tải xong mọi hồ sơ, producer phải dùng lệnh `queue.put(None)`, với `None` là một giá trị biểu diễn nào đó được nhắc đến làm ngừng lại vòng lặp để Consumer sẽ không lấy hồ sơ rỗng nào ra cả (`None` như là một giá trị null).
5. Trong hàm `consumer()`, sẽ có hàm `queue.get()`, consumer sẽ phải lấy hồ sơ của người dùng ra từ hàng đợi và hàm này sẽ phục vụ điều đó bằng cách lấy phần tử đứng trước nhất trong hàng đợi. Thời gian mili giây mà producer lấy một thứ gì đó trong hàng đợi, thì consumer sẽ tự động 'thức dậy' để xử lý nó.

Kết quả của nó cũng tương tự như chuỗi coroutine với khoảng thời gian tương tự:

```
$ python3 asyncioEx4.py
Producer: fetching user by user_id=1...
Producer: fetching user by user_id=2...
Producer: fetching user by user_id=3...
Producer: fetched user with user_id=2 (done in 0.5s)
Consumer: retrieving posts for User2...
Producer: fetched user with user_id=1 (done in 1.0s)
Consumer: retrieving posts for User1...
Producer: fetched user with user_id=3 (done in 1.2s)
Consumer: retrieving posts for User3...
Consumer: got 2 posts by User2 (done in 1.8s):
- Post 1 by User2
- Post 2 by User2
Consumer: got 2 posts by User1 (done in 1.6s):
- Post 1 by User1
- Post 2 by User1
Consumer: got 2 posts by User3 (done in 1.5s):
- Post 1 by User3
- Post 2 by User3
Total time: 2.68 seconds.
```

Hình 11: Kết quả mô phỏng quy trình đăng bài đăng bằng hàng đợi bất đồng bộ

Và lưu đồ biểu diễn dòng thời gian là như sau:



Hình 12: Lưu đồ mô phỏng quy trình đăng bài bằng hàng đợi bất đồng bộ

Quá trình đoạn code này cũng mất tổng cộng 2.68 giây, nhưng cách công việc luân chuyển lại khác biệt hoàn toàn:

1. Giây 0.0: 1 Producer bắt đầu làm việc. 3 Consumer đứng vào vị trí sẵn sàng đợi ở Queue.
2. Giây 0.5: Producer xử lý xong USER 2 sớm nhất (mất 0.5s). Producer dùng lệnh `put()` đặt USER 2 vào Queue. Lập tức, Consumer A dùng lệnh `get()` để chụp lấy USER 2 mang đi xử lý bài đăng (việc này sẽ mất 1.8s). Trong lúc đó, Producer bỏ qua queue và tiếp tục đi xử lý các User khác.
3. Giây 1.0: Producer xử lý xong USER 1 (mất 1.0s). Đặt USER 1 vào Queue bằng `put()`. Do Consumer A đang bận, Consumer B sẽ thực hiện lệnh `get()` chụp lấy USER 1 và mang đi tải bài đăng (việc này mất 1.6s).
4. Giây 1.2: Producer hoàn thành nốt USER 3 (mất 1.2s) và ném vào Queue. Consumer C slảy mang đi tải bài đăng (mất 1.5s). Lúc này Producer đặt nốt giá trị `None` lên kệ và tạm dừng.
5. Giây 2.3: Consumer A xử lý xong bài đăng của USER 2 (0.5s làm hồ sơ + 1.8s tải bài = 2.3s). In kết quả ra màn hình.
6. Giây 2.6: Consumer B xử lý xong bài đăng của USER 1 (1.0s + 1.6s = 2.6s).
7. Giây 2.68: Consumer C xử lý xong bài đăng của USER 3 (1.2s + 1.5s = 2.68s) (có sai số mili-giây). Cả 3 Consumer quay lại nhìn lên kệ, thấy `None`, lập tức ngắt vòng lặp và kết thúc chương trình.

6 Các tính năng I/O bất đồng bộ

Các tính năng I/O bất đồng bộ của Python không chỉ giới hạn ở các cấu trúc `async def` và `await`. Chúng bao gồm các công cụ nâng cao khác giúp lập trình bất đồng bộ trở nên nhất quán với các cấu trúc Python thông thường.

6.1 Các Iterator, vòng lặp và Comprehensions

Ngoài việc sử dụng `async` và `await` để tạo coroutine. Python cũng cung cấp cấu trúc `async for` để lặp qua một iterator bất đồng bộ. **Iterator bất đồng bộ cho phép lặp qua dữ liệu được tạo ra bất đồng bộ. Trong khi vòng lặp chạy, nó trả lại quyền điều khiển cho vòng lặp sự kiện để các tác vụ bất đồng bộ khác có thể chạy.**

Một sự mở rộng tự nhiên của khái niệm này là một generator bất đồng bộ. Dưới đây là một ví dụ tạo ra lũy thừa của hai và sử dụng chúng trong một vòng lặp và list comprehension:

```
>>> import asyncio

>>> async def powers_of_two(stop=10):
...     exponent = 0
...     while exponent < stop:
...         yield 2**exponent
...         exponent += 1
...         await asyncio.sleep(0.2) # Simulate some
asynchronous work
...

>>> async def main():
...     g = []
...     async for i in powers_of_two(5):
...         g.append(i)
...     print(g)
...     f = [j async for j in powers_of_two(5) if
not (j // 3 % 5)]
...     print(f)
...

>>> asyncio.run(main())
[1, 2, 4, 8, 16]
[1, 2, 16]
```

Có một sự khác biệt quan trọng giữa các generator, vòng lặp và Comprehensions đồng bộ và bất đồng bộ là các phiên bản bất đồng bộ của chúng không tự động làm cho quá trình lặp diễn ra đồng thời. Hãy tưởng tượng bạn cần bưng 5 đĩa thức ăn ra bàn; nếu sử dụng vòng lặp `async for`, ta vẫn phải bưng lần lượt từng đĩa một, nghĩa là bản thân quá trình lặp vẫn diễn

ra một cách hoàn toàn tuần tự. Giá trị thực sự của vòng lặp này nằm ở chỗ, giữa những lần bùng đĩa, nó cho phép vòng lặp sự kiện tranh thủ chạy xen kẽ các tác vụ khác của nhà hàng khi ta chủ động nhường quyền điều khiển bằng lệnh `await`. Nếu ta thực sự muốn 5 đĩa thức ăn được bùng ra cùng một tích tắc, ta không thể dùng vòng lặp mà bắt buộc phải tạo ra sự đồng thời bằng cách sử dụng hàm gom nhóm `asyncio.gather()`.

Cuối cùng, hệ thống luôn yêu cầu sự nhất quán về mặt công cụ, việc sử dụng `async for` và `async with` là quy định bắt buộc khi bạn làm việc với các trình lặp hoặc trình quản lý ngữ cảnh bất đồng bộ. Nếu ta cố tình dùng lệnh `for` hoặc `with` thông thường cho những việc có tính chất "tạm dừng và chờ đợi" này, sự "lệch pha" sẽ khiến hệ thống không thể xử lý và ngay lập tức gây ra lỗi.

6.2 Lệnh sync bất đồng bộ

Trong lập trình bất đồng bộ, có rất nhiều công việc liên quan đến I/O đòi hỏi ta phải trải qua một quy trình khép kín: Mở tài nguyên để thiết lập, sử dụng chúng để tải dữ liệu, và cuối cùng là dọn dẹp để đóng lại. Phiên bản `async with` ra đời để đóng vai trò như một trợ lý tự động bao bọc và quản lý toàn bộ quy trình phức tạp này. Giá trị cốt lõi của câu lệnh này là nó đảm bảo mọi tài nguyên mạng cơ bản, bao gồm các kết nối và socket, đều sẽ được hệ thống dọn dẹp và giải phóng một cách chuẩn xác, ngay cả khi quá trình chạy gặp sự cố hoặc phát sinh lỗi giữa chừng.

Ví dụ, giả sử cần viết một coroutine để kiểm tra xem một số trang web có đang hoạt động hay không. Để làm điều đó, ta có thể sử dụng `aiohttp`, một thư viện bên thứ ba mà có thể cài đặt bằng cách chạy lệnh `python -m pip install aiohttp` trên dòng lệnh. Dưới đây là một ví dụ thực hiện chức năng cần thiết:

```
>>> import asyncio
>>> import aiohttp

>>> async def check(url):
...     async with aiohttp.ClientSession() as session:
...         async with session.get(url) as response:
...             print(f"{url}: status -> {response.status}")
...

>>> async def main():
...     websites = [
...         "https://realpython.com",
...         "https://pycoders.com",
...         "https://www.python.org",
...     ]
...     await asyncio.gather(*(check(url) for url
in websites))
...
```

```
>>> asyncio.run(main())
https://www.python.org: status -> 200
https://pycoders.com: status -> 200
https://realpython.com: status -> 200
```

Ví dụ này đã minh họa rõ tính năng này thông qua việc sử dụng thư viện bên thứ ba `aiohttp` để kiểm tra trạng thái hoạt động của một danh sách các trang web khác nhau. Có thể hình dung việc gửi yêu cầu mạng này giống như một cuộc gọi điện thoại: Ta cần mở đường dây (`ClientSession`), chờ bên kia tiếp nhận và phản hồi thông tin (`get(url)`), rồi dập máy. Điểm đáng giá nhất của `async with` là nó thực hiện việc mở và đóng kết nối này một cách bất đồng bộ, hoàn toàn không làm tắc nghẽn vòng lặp sự kiện. chương trình của ta có thể tranh thủ luân chuyển làm việc khác trong khoảng thời gian chờ trang web phản hồi thay vì bị ngừng. Đặc biệt, khi ta bọc các tác vụ này lại và chạy bằng lệnh gom nhóm `asyncio.gather()`, chương trình có thể tiến hành kiểm tra đồng thời nhiều trang web cùng lúc, cho phép ta thu thập trạng thái của các URL song song mà không cần phải đợi trang web trước đó hoàn thành.

6.3 Các công cụ asyncio khác

Ngoài `asyncio.run()`, module `asyncio` cũng đã sử dụng một vài hàm cấp gói khác, chẳng hạn như `asyncio.gather()` và `asyncio.get_event_loop()`. Có thể sử dụng `asyncio.create_task()` để lên lịch thực thi một đối tượng coroutine, sau đó gọi hàm `asyncio.run()` như thường lệ, **hàm `create_task()` để bọc một đối tượng có thể chờ đợi thành một tác vụ (task) cấp cao hơn**. Khác với lệnh `await` thông thường sẽ tạm dừng ngay việc thực thi của tiến trình gọi lệnh để chờ kết quả, việc sử dụng `create_task()` giống như ta giao phó công việc cho vòng lặp sự kiện tự động lên lịch chạy đồng thời trong nền.

```
>>> import asyncio

>>> async def coro(numbers):
...     await asyncio.sleep(min(numbers))
...     return list(reversed(numbers))
...

>>> async def main():
...     task = asyncio.create_task(coro([3, 2, 1]))
...     print(f"{type(task) = }")
...     print(f"{task.done() = }")
...     return await task
...

>>> result = asyncio.run(main())
type(task) = <class '_asyncio.Task'>
task.done() = False
>>> print(f"result: {result}")
```

```
result: [1, 2, 3]
```

Tuy nhiên, có một chi tiết cần đặc biệt lưu ý trong đoạn mã, **nếu ta tạo ra các tác vụ bằng lệnh này nhưng không sử dụng await hoặc gather() để bọc và thu thập chúng**, thì khi hàm chính `main()` kết thúc, toàn bộ các tác vụ chạy ngầm được tạo thủ công đó sẽ bị hủy bỏ do vòng lặp sự kiện đóng lại. Do đó, quy tắc bắt buộc là ta phải sử dụng `await` cho tất cả các tác vụ nếu muốn đảm bảo chúng thực sự được chạy đến khi hoàn thành.

Khi cần quản lý nhiều tác vụ cùng lúc, hàm `gather()` đóng vai trò gom trọn một tập hợp các coroutine vào một đối tượng duy nhất và yêu cầu hệ thống phải chờ tất cả các tác vụ này hoàn thành. Kết quả thu được từ hàm này mang tính xác định tuyệt đối, nghĩa là danh sách kết quả trả về sẽ được sắp xếp tương ứng đúng với thứ tự của các tác vụ đầu vào ban đầu mà ta đã truyền vào. Ví dụ như sau:

```
>>> import time

>>> async def main():
...     task1 = asyncio.create_task(coro([10, 5, 2]))
...     task2 = asyncio.create_task(coro([3, 2, 1]))
...     print("Start:", time.strftime("%X"))
...     result = await asyncio.gather(task1, task2)
...     print("End:", time.strftime("%X"))
...     print(f"Both tasks done: {all((task1.done(), task2.done())
... )}")
...     return result
...

>>> result = asyncio.run(main())
Start: 14:38:49
End: 14:38:51
Both tasks done: True

>>> print(f"result: {result}")
result: [[2, 5, 10], [1, 2, 3]]
```

Hàm `gather()` ở đây chờ toàn bộ kết quả của toàn bộ tập hợp các coroutine mà ta truyền vào. Thứ tự kết quả từ `gather()` là xác định và tương ứng với thứ tự các đối tượng có thể đợi ban đầu được truyền vào.

Ngoài ra cũng có thể lặp qua `asyncio.as_completed()` để lấy các tác vụ khi chúng hoàn thành. Hàm này trả về một trình lặp đồng bộ cung cấp các tác vụ khi chúng kết thúc. Bên dưới, kết quả của `coro([3, 2, 1])` sẽ có sẵn trước khi `coro([10, 5, 2])` hoàn thành, điều này không xảy ra với hàm `gather()`:

```
>>> async def main():
...     task1 = asyncio.create_task(coro([10, 5, 2]))
...     task2 = asyncio.create_task(coro([3, 2, 1]))
```

```

...     print("Start:", time.strftime("%X"))
...     for task in asyncio.as_completed([task1, task2]):
...         result = await task
...         print(f'result: {result} completed at
{time.strftime("%X")}')
...     print("End:", time.strftime("%X"))
...     print(f"Both tasks done: {all((task1.done(), task2.done()
))}")
...

>>> asyncio.run(main())
Start: 14:36:36
result: [1, 2, 3] completed at 14:36:37
result: [2, 5, 10] completed at 14:36:38
End: 14:36:38
Both tasks done: True

```

Trong ví dụ này, hàm `main()` sử dụng `asyncio.as_completed()`, hàm này **trả về các tác vụ theo thứ tự chúng hoàn thành**, chứ **không phải theo thứ tự chúng được bắt đầu**. Khi chương trình lặp qua các tác vụ, nó sẽ chờ chúng, cho phép kết quả có sẵn ngay lập tức sau khi hoàn thành.

Kết quả là, tác vụ nhanh hơn (`task1`) hoàn thành trước và kết quả của nó được in ra sớm hơn, trong khi tác vụ dài hơn (`task2`) hoàn thành và in ra sau đó. Hàm `as_completed()` rất hữu ích khi ta cần xử lý các tác vụ một cách động khi chúng hoàn thành, điều này giúp cải thiện khả năng phản hồi trong các quy trình làm việc đồng thời.

6.4 Xử lý ngoại lệ bất đồng bộ

Bắt đầu với Python 3.11, ta có thể sử dụng lớp `ExceptionGroup` để xử lý nhiều ngoại lệ không liên quan có thể xảy ra đồng thời. Điều này đặc biệt hữu ích khi chạy nhiều coroutine có thể gây ra các ngoại lệ khác nhau. Ngoài ra, cú pháp `except*` mới giúp ta xử lý nhiều lỗi cùng một lúc. Ví dụ như sau:

```

>>> import asyncio

>>> async def coro_a():
...     await asyncio.sleep(1)
...     raise ValueError("Error in coro A")
...

>>> async def coro_b():
...     await asyncio.sleep(2)
...     raise TypeError("Error in coro B")
...

```

```

>>> async def coro_c():
...     await asyncio.sleep(0.5)
...     raise IndexError("Error in coro C")
...

>>> async def main():
...     results = await asyncio.gather(
...         coro_a(),
...         coro_b(),
...         coro_c(),
...         return_exceptions=True
...     )
...     exceptions = [e for e in results if
isinstance(e, Exception)]
...     if exceptions:
...         raise ExceptionGroup("Errors", exceptions)
...

```

Trong ví dụ này, ta có ba coroutine tạo ra ba loại ngoại lệ khác nhau. Trong hàm `main()`, ta gọi hàm `gather()` với các coroutine làm đối số. Ta cũng đặt đối số `return_exceptions` thành `True` để có thể bắt các ngoại lệ nếu chúng xảy ra.

Tiếp theo, ta sử dụng list comprehension để lưu trữ các ngoại lệ vào một danh sách mới. Nếu danh sách chứa ít nhất một ngoại lệ, thì ta tạo một `ExceptionGroup` cho chúng.

Để xử lý nhóm ngoại lệ này, có thể sử dụng đoạn mã sau:

```

>>> try:
...     asyncio.run(main())
... except* ValueError as ve_group:
...     print(f"[ValueError handled] {ve_group.exceptions}")
... except* TypeError as te_group:
...     print(f"[TypeError handled] {te_group.exceptions}")
... except* IndexError as ie_group:
...     print(f"[IndexError handled] {ie_group.exceptions}")
...
[ValueError handled] (ValueError('Error in coro A'),)
[TypeError handled] (TypeError('Error in coro B'),)
[IndexError handled] (IndexError('Error in coro C'),)

```

Trong đoạn này đã bao bọc lời gọi đến `asyncio.run()` trong một khối `try`. Sau đó, ta sử dụng cú pháp `except*` để **bắt ngoại lệ dự kiến** một cách riêng biệt, mỗi trường hợp sẽ được in thông báo lỗi ra màn hình.

7 Khi nào nên sử dụng I/O bất đồng bộ

Việc sử dụng `async def` cho các hàm thực hiện các thao tác chặn (block operations), chẳng hạn như I/O tệp tiêu chuẩn hoặc các yêu cầu mạng đồng bộ sẽ chặn toàn bộ vòng lặp sự kiện, làm mất đi lợi ích của I/O bất đồng bộ và có thể làm giảm hiệu quả của chương trình. Chỉ sử dụng các hàm `async def` cho các thao tác không chặn (non-blocking operations).

Cuộc chiến giữa I/O bất đồng bộ và đa xử lý không phải là một cuộc chiến thực sự. Ta có thể sử dụng cả hai mô hình cùng nhau nếu muốn. Trên thực tế, đa xử lý sẽ là lựa chọn đúng đắn nếu ta có nhiều tác vụ phụ thuộc vào CPU (CPU-bound-tasks).

Cuộc cạnh tranh giữa I/O bất đồng bộ và đa luồng thì trực tiếp hơn. Đa luồng không đơn giản, và ngay cả trong trường hợp đa luồng có vẻ dễ thực hiện, nó vẫn có thể dẫn đến các lỗi khó truy tìm do điều kiện tranh chấp và sử dụng bộ nhớ, cùng nhiều thứ khác. **Đa luồng cũng có xu hướng mở rộng kém hiệu quả hơn so với I/O bất đồng bộ vì luồng là một tài nguyên hệ thống có số lượng hữu hạn. Việc tạo ra hàng nghìn luồng sẽ thất bại trên nhiều máy hoặc có thể làm chậm mã của ta.** Ngược lại, việc tạo ra hàng ngàn tác vụ I/O bất đồng bộ là hoàn toàn khả thi.

I/O bất đồng bộ phát huy hiệu quả khi ta có nhiều tác vụ ràng buộc bởi I/O mà nếu không sẽ bị chi phối bởi thời gian chờ chặn, chẳng hạn như:

- I/O mạng, cho dù chương trình của bạn hoạt động như máy chủ hay máy khách/
- Thiết kế không máy chủ (serverless), chẳng hạn như mạng ngang hàng (peer-to-peer), đa người dùng như một nhóm trò chuyện.
- Các thao tác đọc/ghi mà ta muốn mô phỏng cách tiếp cận kiểu "gửi đi và quên" (fire-and-forget) mà không cần lo lắng về việc giữ khóa tài nguyên.

Lý do lớn nhất để không sử dụng I/O bất đồng bộ là `await` chỉ hỗ trợ một tập hợp các đối tượng cụ thể định nghĩa một tập hợp các phương thức cụ thể. Ví dụ, nếu ta muốn thực hiện các thao tác đọc bất đồng bộ trên một hệ quản trị cơ sở dữ liệu (DBMS) nhất định, thì ta cần tìm một lớp bao bọc Python cho DBMS đó hỗ trợ cú pháp `async` và `await`.

8 Bài tập

- 8.1. Viết một kịch bản mô phỏng việc tải 3 tệp dữ liệu (File A, B, C) từ một máy chủ. Biết rằng mỗi tệp cần đúng 2 giây để tải xong. Nếu chạy đồng bộ sẽ mất 6 giây. Ta cần phải viết lại bằng mã bất đồng bộ để tổng thời gian tải cả 3 tệp chỉ mất 2 giây. Tên tệp sẽ được gọi là `loadFile.py`, sử dụng khối `__main__` để thực thi. Kết quả nên tương tự như sau:

```
Load file A
Load file B
Load file C
Execute time: 2.00s
```

8.2. Hãy mô phỏng lại quy trình phục vụ của một ứng dụng giao đồ ăn với 2 bước bắt buộc:

- (a) `prepare_order(order_id)`: Đầu bếp chuẩn bị món ăn (mất 1.5 giây).
- (b) `deliver_order(order_id)`: Shipper giao hàng (mất 1 giây).

Ta có 3 đơn hàng (ID: 101, 102, 103). Shipper không thể giao nếu đầu bếp chưa nấu xong (tính tuần tự). Hãy viết chương trình sao cho cả 3 đơn hàng này được xử lý đồng thời trên hệ thống. In ra thời điểm bắt đầu và kết thúc của từng giai đoạn cho mỗi đơn hàng. Tên tệp sẽ được gọi là `shipOrder.py`, sử dụng khối `__main__` để thực thi. Kết quả nên tương tự như sau:

```
Preparing food order: 101
Preparing food order: 102
Preparing food order: 103
Prepared food order: 101
Receiving Order 101
Prepared food order: 102
Receiving Order 102
Prepared food order: 103
Receiving Order 103
Deliver Order 101
Deliver Order 102
Deliver Order 103
Execute time: 2.50s
```

8.3. Ta đang truy vấn thông tin thời tiết từ 3 trạm đo lường khác nhau. Trạm A phản hồi ngẫu nhiên từ 1-3 giây, trạm B từ 0.5-2 giây, trạm C từ 2-4 giây. Khác với việc phải đợi trạm chậm nhất phản hồi để in ra toàn bộ kết quả một lượt, hãy thiết kế chương trình sao cho trạm nào phản hồi xong trước, in ngay kết quả của trạm đó ra màn hình theo đúng thứ tự hoàn thành. Tên tệp sẽ được gọi là `queryWeathers.py`, sử dụng khối `__main__` để thực thi. Kết quả nên tương tự như sau:

```
Querying weather A
Querying weather B
Querying weather C
Successfully querying weather B
Successfully querying weather A
Successfully querying weather C
Execute time: 2.97s
```

Lưu ý: Kết quả có thể khác hoàn toàn, sử dụng `random.seed(444)` trong hàm `__main__` để giữ nguyên một kết quả ngẫu nhiên duy nhất.

8.4. Công ty ABC cần một hệ thống xử lý email nội bộ, phân tách rõ ràng giữa máy chủ nhận yêu cầu và máy khách thực hiện gửi mail, hãy thực hiện những việc sau:

- Viết 1 hàm Producer tạo ra 5 email (ví dụ `email_1` đến `email_5`) cách nhau mỗi 0.2 giây, đặt chúng vào hàng đợi. Cuối cùng đặt một giá trị `None` để báo hiệu kết thúc.
- Viết 2 hàm Consumer tranh nhau lấy email từ hàng đợi để xử lý. Thời gian xử lý mỗi email (mô phỏng gửi đi) là 1 giây.
- In ra màn hình xem Consumer 1 hay Consumer 2 đã gửi email nào.

Tên tệp sẽ được gọi là `emailManagement.py`, sử dụng khối `__main__` để thực thi. Kết quả nên tương tự như sau:

```
Fetching email_1
Fetched email_1
Fetching email_2
Fetched email_2
Consumer 1 retrieves email_1.
Fetching email_3
Fetched email_3
Consumer 2 retrieves email_2.
Fetching email_4
Fetched email_4
Fetching email_5
Fetched email_5
Consumer 1 sends email_1.
Consumer 1 retrieves email_3.
Consumer 2 sends email_2.
Consumer 2 retrieves email_4.
Consumer 1 sends email_3.
Consumer 1 retrieves email_5.
Consumer 2 sends email_4.
Consumer 1 sends email_5.
Execute time: 3.20s
```

8.5. Có 3 dịch vụ gọi là `Database`, `Cache_Server`, và `Payment_Gateway`. Hãy thực hiện một chương trình quét 3 dịch vụ này sao cho:

- `Database` và `Cache_Server` chạy bình thường (trả về trạng thái 200 OK sau 1 giây).
- `Payment_Gateway` bị lỗi, hãy ép nó sinh ra một ngoại lệ `Connection Error` sau 1.5 giây.

Nhiệm vụ chính là khởi chạy cả 3 tiến trình quét này đồng thời, nhưng không được để ngoại lệ của `Payment` phá sập toàn bộ chương trình. Phải hứng được lỗi này, báo cáo nó, và vẫn thu thập đủ kết quả 200 OK của

2 dịch vụ kia. Tên tệp sẽ được gọi là `findServices.py`, sử dụng khối `__main__` để thực thi. Kết quả nên tương tự như sau:

```
Scan for service Database
Scan for service Cache_Server
Scan for service Payment_Gateway
Database: 200 OK
Cache_Server: 200 OK
Caught exception: Can't connect to Payment_Gateway
Execute time: 1.50s
```

Gợi ý: Khi dùng `asyncio.gather()`, hãy truyền tham số này để trợ giúp (`return_exceptions=True`). Điều này giúp hàm không bị crash, mà xem ngoại lệ như một kết quả hợp lệ để trả về danh sách.

8.6. Mô phỏng đường ống (pipeline) thu thập dữ liệu web toàn diện như sau:

- Viết một Generator bất đồng bộ (`async def generate_urls()`) chứa từ khóa `yield`. Generator này sẽ sinh ra 4 địa chỉ URL liên tiếp (độ trễ 0.1s mỗi URL).
- Một luồng Producer sẽ dùng `async for` để nhận các URL này từ Genetator, sau đó nhét ngay vào một `asyncio.Queue()`.
- Hai luồng Consumer sẽ lấy URL ở `Queue`. Khi lấy được URL, Consumer phải sử dụng khối `async with` (tự định nghĩa một class ngữ cảnh giả lập việc tải HTML an toàn) để mô phỏng mở kết nối.
- Sau khi có HTML (dữ liệu giả), Consumer đó tiếp tục xâu chuỗi (chaining) gọi một hàm `parse_data(html)` để trích xuất thông tin.

Dữ liệu HTML sẽ là như sau:

```
<!DOCTYPE HTML>
<html>
  <head>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Hello from {page_id}</h1>
  </body>
</html>
```

ên tệp sẽ được gọi là `asyncWebContext.py`, sử dụng khối `__main__` để thực thi. Kết quả nên tương tự như sau:

```
Đưa vào hàng đợi URL http://example.com/page_1
Mở kết nối mạng... (http://example.com/page_1)
Đưa vào hàng đợi URL http://example.com/page_2
Mở kết nối mạng... (http://example.com/page_2)
Đưa vào hàng đợi URL http://example.com/page_3
```

```
Đưa vào hàng đợi URL http://example.com/page_4
Consumer 1 đã trích xuất thành công: Hello from page_1!

Đóng kết nối an toàn (http://example.com/page_1)
Mở kết nối mạng... (http://example.com/page_3)
Consumer 2 đã trích xuất thành công: Hello from page_2!

Đóng kết nối an toàn (http://example.com/page_2)
Mở kết nối mạng... (http://example.com/page_4)
Consumer 1 đã trích xuất thành công: Hello from page_3!

Đóng kết nối an toàn (http://example.com/page_3)
Consumer 2 đã trích xuất thành công: Hello from page_4!

Đóng kết nối an toàn (http://example.com/page_4)
Execute time: 1.20s
```

Gợi ý: Đối với việc tạo một Context Manager bất đồng bộ tùy chỉnh (để dùng với `async with`), hãy tạo một class và định nghĩa hai hàm `__aenter__()` (để khởi tạo tài nguyên) và `__aexit__()` (để dọn dẹp tài nguyên). Hãy mô phỏng việc "Mở kết nối mạng..." và "Đóng kết nối an toàn" bên trong hai hàm này.